Engineering Applications of Artificial Intelligence 26 (2013) 1698–1706

Contents lists available at SciVerse ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Brief paper

Solving the forward kinematics problem in parallel robots using Support Vector Regression

Antonio Morell [a,*], Mahmoud Tarokh [b], Leopoldo Acosta [a]

[a] Department of Ingeniería de Sistemas y Automática y Arquitectura y Tecnología de Computadores (ISAATC), Universidad de La Laguna, 38203 La Laguna, Tenerife, Spain
[b] Department of Computer Science, San Diego State University, San Diego, CA 92182-7720, USA

ARTICLE INFO

Article history:
Received 14 June 2012
Received in revised form
28 February 2013
Accepted 18 March 2013
Available online 3 May 2013

Keywords:
Parallel robots
Stewart platform
Forward kinematics
Real-time
Support vector machines
Spatial decomposition

ABSTRACT

The Stewart platform, a representative of the class of parallel manipulators, has been successfully used in a wide variety of fields and industries, from medicine to automotive. Parallel robots have key benefits over serial structures regarding stability and positioning capability. At the same time, they present challenges and open problems which need to be addressed in order to take full advantage of their utility. In this paper, we propose a new approach for solving one of these key aspects: the solution to the forward kinematics in real-time, an under-defined problem with a high-degree nonlinear formulation, using a popular machine learning method for classification and regression, the Support Vector Machines. Instead of solving a numerical problem, the proposed method involves applying Support Vector Regression to model the behavior of a platform in a given region or partition of the pose space. It consists of two phases, an off-line preprocessing step and a fast on-line evaluation phase. The experiments made have yielded a good approximation to the analytical solution, and have shown its suitability for real-time application.

© 2013 Elsevier Ltd. All rights reserved.

1. Introduction

A few years after the term *robot* was coined, the first industrial parallel robot, a spray painting machine, was patented by Pollard (1940). Extensively used as manipulators in a wide variety of fields, e.g. medicine, optics, astronomy and many industries like aerospace, automotive and aviation, parallel robots have attracted the attention of many researchers in recent decades. They can be classified as closed-loop mechanisms, in contrast to serial robots, which are usually open-loop kinematics chains. Instead, they consist of two frames connected by means of active links, such as prismatic actuators, sometimes called *legs*. The advantages of this type of structure include a greater rigidity and positioning capability, good dynamic performance and high load carrying capacity, which make them very attractive in many applications and fields. Notable examples of parallel manipulators are the *Delta* robot (Clavel, 1988), the *Tricept* (Siciliano, 1999) and the Gough–Stewart platform. Presented by Gough (1956–1957) as a tyre testing machine, it is one of the most popular parallel manipulators. It gained popularity as a flight simulator (Stewart, 1965–1966) and is commonly known as the *Stewart platform*. The first design as a manipulator system was presented by McCallion and Pham (1979) as an assembly workstation. A detailed and very informative review by Dasgupta and Mruthyunjaya (2000) provides an extensive account of some relevant aspects of the Stewart platform.

In this paper we present a novel method for solving the Forward Kinematics Problem (FKP), still a relevant topic for some types of parallel manipulators, e.g., those with joint offsets. For such robots, unlike the inverse kinematics problem, the FKP lacks a closed-form mathematical solution. It is an under-defined problem with a high-degree of nonlinearity in which the solution in most cases is not unique. Although kinematics is one of the most studied topics of parallel robots, the FKP continues to gain interest, especially in terms of those methods which can solve it in real-time. This is essential for a platform characterization and its closed-loop control to know the position and orientation (*pose*) by means of the length of the linear actuators attached to the joints.

A review of the literature shows that the FKP in parallel robots has been solved in recent years using numerical as well as approximate methods and strategies. Liu et al. (1993) describe an analytical solution for the generalized configuration, i.e., 6-SPS (Spherical–Prismatic–Spherical), whereas some authors propose a simplification of the model (Nanua et al., 1990), which is solved with numerical methods like Newton–Raphson, or use of an interval analysis (Merlet, 2004) to work out the solution. However, these methods lack generalization, since they are proposed for particular types of platform. Other methods develop a solution

* Corresponding author. Tel.: +34 922 31 82 87; fax: +34 922 31 82 88.
E-mail addresses: amorell@isaatc.ull.es (A. Morell), tarokh@cs.sdsu.edu (M. Tarokh), lacosta@ull.es (L. Acosta).

0952-1976/$ - see front matter © 2013 Elsevier Ltd. All rights reserved.
http://dx.doi.org/10.1016/j.engappai.2013.03.011

to the FKP by directly applying a numerical method (Dieudonne et al., 1972) like Newton–Raphson. For instance, Dalvand and Shirinzadeh (2011, 2012) proposed an algorithm which yields solutions by means of an iterative method for a platform with joint offsets, a 6-RRCRR (Rotational–Rotational–Cylindrical–Rotational–Rotational) parallel manipulator, which does not have a closed-form solution for the FKP. In general, these techniques can achieve accurate solutions with enough iterations of the algorithms, but may have convergence problems and high computational requirements. Furthermore, there are other solutions that add rotary type sensors or extra links (Merlet, 1993; Chen and Fu, 2006) to obtain additional information, in order to aid and simplify the algorithms. However, these approaches may be difficult to generalize and in some cases will not be applicable due to structural or mechanical constraints. In addition, approximate solution strategies have been successfully applied, mainly by means of Artificial Intelligence methods like neural networks, in order to study kinematics of serial robots (Gao et al., 2010; Karlik and Aydin, 2000; Köker, 2005; Chiddarwar and Babu, 2010) and parallel robots (Parikh and Lam, 2009; Tarokh, 2007). These strategies are more suited to real-time applications and obtain good enough approximations for a wide variety of fields.

This paper presents a spatial decomposition method for obtaining accurate solutions in real-time for the FKP of Stewart platforms using a popular machine learning method, the *Support Vector Machines* (SVMs), as the regression model. Firstly, the method is applied to a 6-SPS parallel manipulator for which an analytical solution exists. In order to verify its correctness and efficiency, the yielded results are compared with the polynomial curve fitting method proposed by Tarokh (2007), and against the exact analytical solution for a generalized Stewart platform presented by Liu et al. (1993). Secondly, a similar experiment is conducted with a real parallel manipulator, the M-850 hexapod by *Physik Instrumente*.

This paper is organized as follows. Kinematics in parallel robots are discussed in Section 2. The spatial decomposition method is introduced in Section 3, followed by the description of the classification procedure in Section 4. Then, the forward kinematics modeling with SVR machines and the *on-line* evaluation procedure are detailed in Section 5. Finally, results of different experiments are discussed in Section 6, and the main conclusions are summarized in Section 7.

2. On kinematics of parallel robots

2.1. The 6-SPS general Gough–Stewart platform

The theory of serial–parallel duality, which highlights the qualitative distinctions between serial and parallel manipulators, states that in both position and velocity there is a symmetric relationship in the forward and inverse cases (Collins and Long, 1995). In contrast to the simple forward kinematics and complicated inverse kinematics of serial manipulators (requiring the solution of a system of nonlinear equations), parallel manipulators exhibit more or less straightforward inverse kinematics and a challenging solution for the forward kinematics problem.

The generalized Gough–Stewart platform is the most celebrated manipulator in the entire class of parallel robots. It has found a central status in the literature due to the fact that it exhibits the serial–parallel duality in the most prominent manner.

As shown in Fig. 1, the generalized configuration is composed of two platforms and a set of linear actuators, typically six, often called *legs*. Consider a fixed base B, with a coordinate frame XYZ attached to it, and another coordinate frame xyz fixed to the top platform A. Let us define the *link space* as the 6-D space consisting

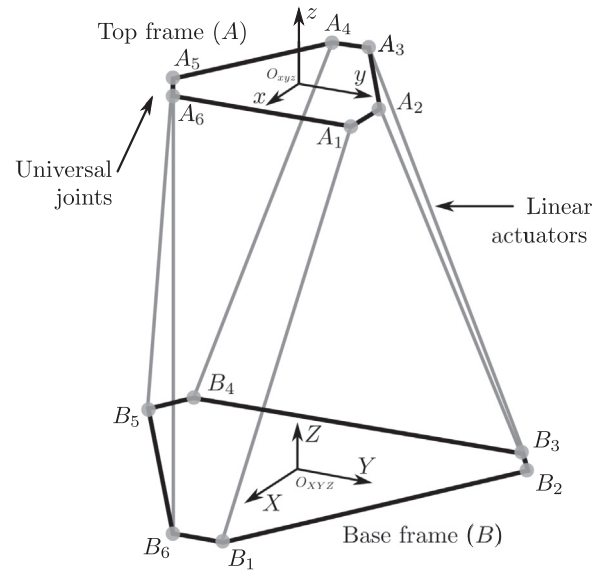

Fig. 1. The general Gough–Stewart platform, a class of parallel robots.

of the value of the length of each leg. A *link vector* represents a position in the link space as a 6-D vector $l=[l_1,l_2,\ldots,l_6]^{\top}$. Similarly, the *pose space* is the 6-D space that represents the position and orientation of the top platform (or end effector), as a combination in 3-D Cartesian coordinates of position $[x,y,z]$ and 3-D orientation angles $[\alpha,\beta,\gamma]$. A *pose* is defined by the vector $p=[x,y,z,\alpha,\beta,\gamma]^{\top}$.

Let d denote the displacement vector of the frame $[x,y,z]$ relative to the frame XYZ, let B_i be the position of the i-th link (*leg*) attached to the base relative to XYZ, and similarly let A_i be the position of the i-th link attached to the top platform with respect to $[x,y,z]$. Finally let R be the 3×3 rotation matrix that defines the rotating angles of the frame $[x,y,z]$ with respect to the frame XYZ. Since there is no joint offsets, the length of each link connecting the base to the top can be written as

$$\begin{aligned} l_i &= \sqrt{\|RA_i+d-B_i\|} \quad i=1,2,\ldots,6 \\ &= f_i(p), \end{aligned} \tag{1}$$

where $f_i(p)$ is a known function and p is the top platform pose. It is noted that A_i and B_i are known for a given hexapod. Furthermore, specification of the pose, i.e., position and orientation of the top platform, determines the rotation matrix R and displacement vector d.

For parallel robots, the forward kinematics problem can be stated as follows: Given a link vector l, and the position of link attachments to the top and base platforms, A_i and B_i (with $i=1,2,3,4,5,6$), respectively, find the set of all possible poses p that satisfy (1).

2.2. A platform with joint offsets: the 6-RRCRR case

This kind of hexapod is convenient in some situations, for example when there are manufacturing constraints, since universal joints are more complex to produce. Fig. 2 shows a diagram for a given leg. It is composed of two revolute joints, separated by an offset given by P_J, a prismatic joint, a passive cylindrical joint, and similarly, another two revolute joints with the same offset. For the i-th leg, the joint variables are q_{ki}, with $k=1,2,3,4,5,6$. This configuration leads to more complicated kinematics equations, since dependency between joint variables exists. An inverse kinematics solution for this parallel manipulator is the distance between joints q_{1i} and q_{6i}, i.e., the Euclidean norm of vector q_{3i},

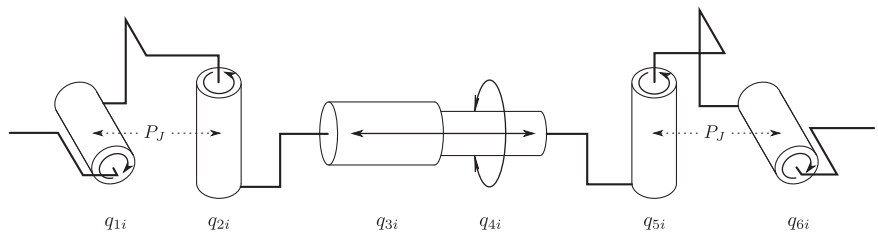

Fig. 2. Leg diagram of the 6-RRCRR model.

as follows:

$$q_{3i} = \sqrt{{}^{B_i}X_{q_{3i}}^{2B_i} + {}^{B_i}Y_{q_{3i}}^{2B_i} + {}^{B_i}Z_{q_{3i}}^{2B_i}} \quad (\forall i = 1, \ldots, 6), \tag{2}$$

where

$$\begin{aligned}
{}^{B_i}X_{q_{3i}}^{2B_i} &= X + P_A(C(\theta_A)(C(\beta)C(\gamma) + S(\alpha)S(\beta)S(\gamma)) - S(\theta_A)(C(\beta)S(\gamma) \\
&\quad - C(\gamma)S(\alpha)S(\beta))) - P_B C(\theta_B) + P_J S(q_{6i})(C(\rho_A)(C(\theta_A)(C(\beta)S(\gamma) \\
&\quad - C(\gamma)S(\alpha)S(\beta)) + S(\theta_A)(C(\beta)C(\gamma) + S(\alpha)S(\beta)S(\gamma))) \\
&\quad + S(\rho_A)(C(\theta_A)(C(\beta)C(\gamma) + S(\alpha)S(\beta)S(\gamma)) - S(\theta_A)(C(\beta)S(\gamma) \\
&\quad - C(\gamma)S(\alpha)S(\beta)))) - P_J S(q_{1i})(C(\rho_B)S(\theta_B) + C(\theta_B)S(\rho_B)) \\
&\quad - P_J C(q_{1i})S(\phi_B)(C(\rho_B)C(\theta_B) - S(\rho_B)S(\theta_B)) + P_J C(\alpha)S(\beta)C(q_{6i})
\end{aligned}$$

$$\begin{aligned}
{}^{B_i}Y_{q_{3i}}^{2B_i} &= Y - P_B S(\theta_B) + P_A(C(\alpha)C(\gamma)S(\theta_A) + C(\alpha)C(\theta_A)S(\gamma)) \\
&\quad + P_J S(q_{1i})(C(\rho_B)C(\theta_B) - S(\rho_B)S(\theta_B)) \\
&\quad - P_J S(\alpha)C(q_{6i}) - P_J S(q_{6i})(C(\rho_A)(C(\alpha)C(\gamma)C(\theta_A) - C(\alpha)S(\gamma)S(\theta_A)) \\
&\quad - S(\rho_A)(C(\alpha)C(\gamma)S(\theta_A) \\
&\quad + C(\alpha)C(\theta_A)S(\gamma))) - P_J C(q_{1i})S(\phi_B)(C(\rho_B)S(\theta_B) + C(\theta_B)S(\rho_B))
\end{aligned}$$

$$\begin{aligned}
{}^{B_i}Z_{q_{3i}}^{2B_i} &= Z - H_B - P_A(C(\theta_A)(C(\gamma)S(\beta) - C(\beta)S(\alpha)S(\gamma)) - S(\theta_A)(S(\beta)S(\gamma) \\
&\quad + C(\beta)C(\gamma)S(\alpha))) - P_J S(q_{6i})(C(\rho_A)(C(\theta_A)(S(\beta)S(\gamma) \\
&\quad + C(\beta)C(\gamma)S(\alpha)) + S(\theta_A)(C(\gamma)S(\beta) - C(\beta)S(\alpha)S(\gamma))) \\
&\quad + S(\rho_A)(C(\theta_A)(C(\gamma)S(\beta) - C(\beta)S(\alpha)S(\gamma)) - S(\theta_A)(S(\beta)S(\gamma) \\
&\quad + C(\beta)C(\gamma)S(\alpha)))) - P_J C(\phi_B)C(q_{1i}) + P_J C(\alpha)C(\beta)C(q_{6i}).
\end{aligned} \tag{3}$$

As can be seen, there is a dependency with joint angles q_{1i} and q_{6i}, which leads to a set of nonlinear equations. For further details, please see Dalvand and Shirinzadeh (2011, 2012), where the kinematics of this hexapod was studied as a part of a skull-base surgery system. In Section 6 we compare the results yielded by our method with the results reported in that work.

3. Spatial decomposition and data generation

As described by Tarokh (2007), the spatial decomposition method consists of an *off-line* preprocessing phase and a fast *on-line* evaluation phase. In the off-line phase, the method decomposes the link space into small rectangloid *cells*. Using (1), it assigns random values within the range of interest to the pose p and computes the link values l_i. Data points that fall into a cell in the link space have corresponding points in a *region* of the pose space. The relationship between data points in a cell and its corresponding region must be modeled so that when a given set of link values is specified, the sets of all pose values can be computed during the on-line phase. The parameters of this relationship are stored in the form of a look-up table. The parameters are then retrieved during the on-line phase.

The method described by Tarokh (2007) for modeling the relationship involves fitting a set of polynomial equations to obtain a parametric model which approximates the forward kinematics solution. In this paper we propose a different approach using a Support Vector Regression (SVR) machine to relate the platform pose in each *region* to the link vector in the corresponding *cell* of the link space. One of the advantages of SVMs is their ability to easily fit complex nonlinear functions. However, there is a tradeoff between the size of the dataset and the parameters used in terms of training time and accuracy: large training sets will not significantly improve the accuracy of the solutions and will add a time penalty. At the same time, fine tuning the parameters will improve the solutions, but will worsen the training time as well. An example of this behavior is given in Section 6. Therefore, the size of the cells in the link space provides a tradeoff, because it affects the complexity and the number of the machines to be tuned. However, SVR machines allow for fewer link and pose space divisions, since they can model more complex data. Specifically, in the given numerical example described in Section 6, the link and pose space were divided by only 64 regions without accuracy loss, compared with the 15 625 needed by the polynomial method (Tarokh, 2007).

The number of divisions along an axis of the link space can be described by

$$N_i = \frac{l_{i,max} - l_{i,min}}{l_c}, \quad i = 1, 2, \ldots, 6, \tag{4}$$

where l_c is the side length of a cell, which determines its size.

Now each region in the pose space and its corresponding link cell must be populated with a meaningful number of pose and link data points, so that the SVR models can be successfully trained. Randomly generated pose is a better choice than regularly spaced intervals for yielding a valid dataset. The latter will not produce uniform link space coverage due to the nonlinear mapping of the forward kinematics. We have chosen a pseudorandom number generator that belongs to the TGFSR (*Twisted Generalized Feedback Shift Register*) family, introduced by Panneton et al. (2006), called WELL (*Well Equidistributed Long-period Linear*), a class of robust generators with very good equidistribution, periods, speed and other interesting properties. Each generated pose must be tested to discard those that are invalid. A pose is valid if its corresponding link lengths are within the feasible range, and if they do not violate any platform-specific mechanical constraints, e.g., joint limits or link collision.

A cell of the link space is mapped into a region of the pose space, that can be identified by a unique index, as follows. For a given link configuration $L_k = (l_{k1}, l_{k2}, l_{k3}, l_{k4}, l_{k5}, l_{k6})^T$, the index is obtained by

$$C_k = \left(\sum_{i=1}^{6} \left\lfloor \frac{l_{ki} - l_{i,min}}{l_c} \right\rfloor (N_i)^{i-1} \right) + 1, \tag{5}$$

where $l_{i,min}$ is the minimal length or position of the i-th link and N_i is the number of cells in each dimension, given by (4). Thus, the valid generated poses can be associated with a unique link space cell and stored based on their region index in order to be processed in the next step.

4. Classification

The generated data form clusters in the pose space, representing different solutions over a specific region. These clusters must be classified before starting the SVR training, in order to represent a more accurate model. In the example shown in Fig. 3 they appear partially overlapped because of the 2D representation, but in other cases they will look like a more homogeneous cloud.

Depending on the classification method, the number of clusters to identify, K_c, is a required input parameter. It partially depends on the structure of the parallel robot. Irregular shaped platforms with more leg attachment points will have more solutions (poses).

The method chosen to estimate the number of clusters for each region starts by fitting a Gaussian Mixture model using an *Expectation Maximization* (EM) algorithm (Dempster et al., 1977) with maximum likelihood estimates. A measure for the relative fitness of a statistical model called *Akaike Information Criterion* (AIC) is then applied. Developed by Akaike (1974), it is widely used in model selection because it is an estimate of the expected Kullback–Leibler information of a fitted model. A simple strategy to apply this criterion is by an iterative process, where the Gaussian Mixture model is fitted with k components, and where k iterates in $[1, \ldots, k, \ldots, K_{max}]$, i.e., the maximum expected value for K_c, which should be set according to the platform structure. In each iteration the AIC, a dimensionless quantity, is calculated by the following expression:

$$AIC = 2P - 2\ln(L), \tag{6}$$

where P is the number of parameters in the statistical model and L is the maximized value of the likelihood function for the estimated model, obtained with the EM algorithm.

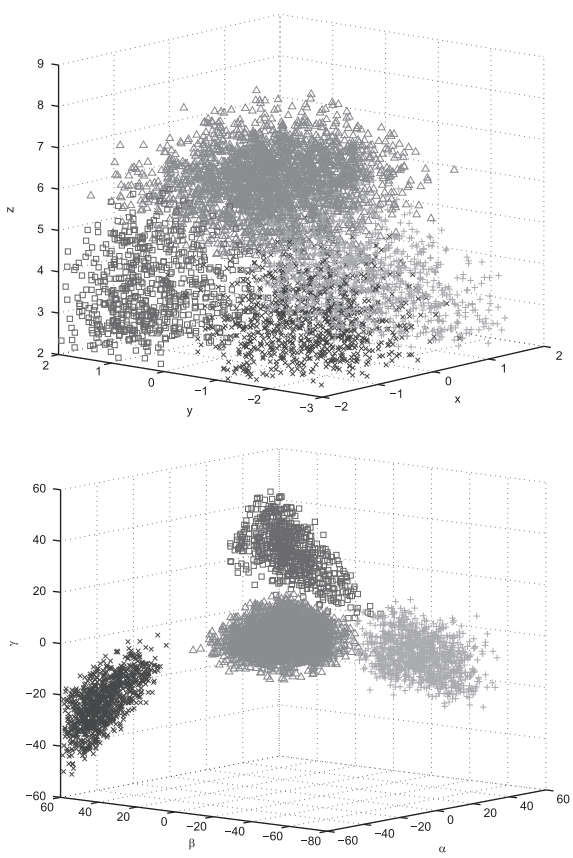

Fig. 3. 3D representation of region 1: first—position subspace; second—orientation subspace.

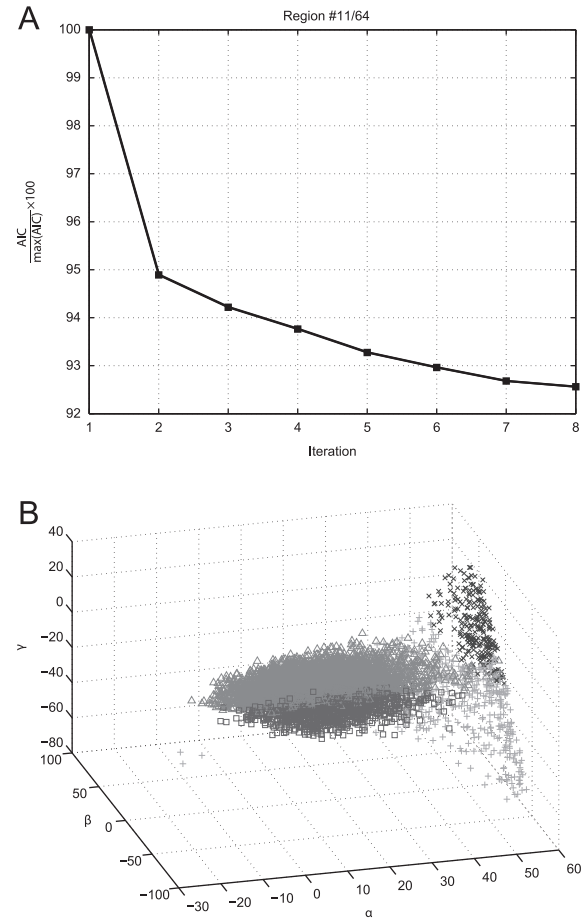

Fig. 4. First—AIC value for each iteration; second—orientation subspace. The cutoff is at the midpoint.

Given the values after each iteration, K_c can be heuristically selected. We have found that a procedure like the *Elbow method* does not guarantee the most appropriate guess of the number of clusters. For example, as illustrated in Fig. 4 (where we have plotted the orientation subspace for simplicity), the presence of an *elbow* point between the second and third iteration suggests a $K_c = 2$. The values are shown as a percentage, normalized by their maximum value, which happens at the first iteration. Whereas the cloud is homogeneous, the points are spread across the region, so it seems appropriate to divide them into more than the two clusters that the Elbow method suggests. One possible approach to automate the selection consists of setting a threshold to find a cutoff iteration from which increasing the number of clusters does not significantly improve the fit. If that iteration is greater than the elbow point, we have found that selecting the midpoint gives a good guess for K_c.

The main advantage of this procedure is that we can calculate the posterior probability of each region point for each cluster from the fitted Gaussian Mixture model. Hence, in order to classify each pose we only need to find the cluster in which its posterior probability is maximum. Furthermore, thanks to the SVR modeling capabilities, there are fewer regions to be classified, 64 instead of 15 625 in the example described in Section 6.

5. FK modeling and evaluation

After the clustering step, the data are ready to be used as training samples. However, the data must be preprocessed before

starting the training process. First, removing outliers could significantly reduce the training time of the related SVR machine. To detect and remove them we have used the Multivariate Outlier Detection algorithm presented by Peña and Prieto (2001). Second, link data must be scaled (normalized) to avoid numerical difficulties and the adverse effect of greater numeric ranges against those which happen to be smaller. Both training and testing data must be scaled with the same method and range, e.g., [−1,1] or [0,1]. In the experiments described in the next section we have used the former to normalize the input data, which is now suitable for being used as SVR training samples.

Support Vector Machines were formally introduced as a pattern recognition tool by Vapnik and his co-workers (Boser et al., 1992), and a few years later they were extended to solve regression problems, constructing multidimensional splines and solving linear operator (Vapnik, 1995; Vapnik et al., 1996). As a statistical learning tool, SVMs were developed with a greater ability to generalize. When solving pattern recognition problems, a SVM is able to find in a feature space a decision rule with good generalization, i.e., the optimal separating hyperplane, by selecting a small subset of the training data, called the Support Vectors (SVs). It can be shown that optimal separation of the SVs is equivalent to the optimal separation of the entire data (Vapnik et al., 1996). Maximizing the margin between classes and training samples is an alternative to other optimizing cost function methods, e.g., mean square error. This provides some important features to SVMs, like automatic capacity tuning of the classification function, the representation of the data relevant for the classification by a small subset (data compression), and uniqueness of the solution. Hence, SVMs can be used to summarize the information contained in a dataset. The experiments carried out in this work prove that using SVR machines to model the behavior of the platform in a given region yields results good enough for a wide variety of applications.

A SVR is a multi-input single-output system, which supports a very large number of inputs as data features, while the output or target value is predicted with a decision function. Since one trained machine will output one predicted value, each pose prediction will need a trained SVR model for each dimension $[x, y, z, \alpha, \beta, \gamma]$ using the same data instances. Therefore, a given cluster will need a total of six trained SVR machines in order to obtain a full pose estimation. Fig. 5 shows a description of the implementation. The training parameters will be described in Section 6.

The training algorithm for a given SVR machine requires an input vector (p_i, l_k), where $l_k \in \Re^6$ is the k-th link configuration vector, with $k=1,\ldots,m$, and $p_i \in \Re$ is the corresponding value of the i-th pose vector parameter. The algorithm finds the parameters of a decision function $F(l)$ using these training samples

$$F(l) = \langle w, l_k \rangle + u. \tag{7}$$

The goal of the algorithm is to find the parameter w and the bias u that represent the optimal separating hyperplane, which model the pose parameter p_i in the given region.

The decision function has a dual space representation, which reduces the number of computations required to train the classifier (Boser et al., 1992)

$$F(l) = \sum_{k=1}^{m} \alpha_k K(l_k, l) + u, \tag{8}$$

where the coefficients α_k are the parameters found by the algorithm, l_k are the training samples, with m the number of samples. $K(l_k, l)$ is the kernel function, which allows operations to be performed in the input space rather than in a high dimensional feature space. Extending the formulation to the *dual form* allows the application of different kernel functions, such as polynomials, radial basis functions and certain sigmoid functions.

As a regression tool, a SVR employs an alternative loss function, which includes a distance measure. Vapnik (1995) proposed the *ε-insensitive loss function*, a more sophisticated penalty tool, which defines a region between $\hat{p}_i$, the predicted value, and p_i, the actual value. Only when $|p_i - \hat{p}_i| \geq \varepsilon$ does the function add one of two slack variable penalties, depending on whether they lie *above* or *below* a certain region called the *ε-insensitive tube*. The ε parameter has an effect on the smoothness of the response and it affects the number of Support Vectors, i.e., the generalization capability.

To store the data of the trained models, the method needs to populate an indexed data structure or *lookup table* in the off-line phase, which should allow fast indexation. Given that we can apply (5) to obtain the address of the k-th element of the input vector, the structure can be a sequence of elements that contains the trained model parameters. The input link data in the on-line phase must be scaled according to the training data to obtain a valid prediction, so the scaling information used for training is also stored. This completes the off-line phase, which only needs to be executed once for a given parallel robot.

Given a link vector l, the address of the lookup table where the model parameters are stored can be obtained by applying (5). Then, after scaling the link coordinates, each pose space parameter is predicted using l. The time to retrieve the six model parameters and generate the estimation is negligible, as will be described in the next section.

The mean error obtained after the experiments is small enough to satisfy a wide variety of applications, but if needed, the accuracy of the approximation can be improved by a correction step. Let $\hat{p}$ be the pose parameters estimated for the l input vector after the on-line phase. Applying the inverse kinematics expression (1) yields an estimation of the link lengths $\hat{l}$ associated with the pose estimation $\hat{p}$. The difference between the target and the estimated link lengths can be calculated as $\Delta l = (l - \hat{l})$, and can be added to l to obtain a new target link configuration $\lambda = l + \Delta l$. Finally, solving the forward kinematics of the corrected link lengths λ will give an improved approximation of the initial link vector l.

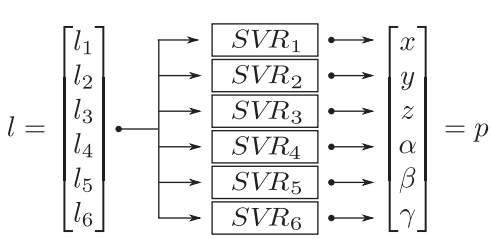

Fig. 5. SVR models corresponding to each pose space parameter for a given cluster.

6. Results and discussion

6.1. Experimental setup

The experiments were made on a workstation with an Intel Xeon CPU E5440@2.83 GHz, 16 GB of RAM, with Linux kernel version 2.6.24-24-generic and MATLAB R2011B, both 64-bit versions. The most time-consuming steps are the classification and the SVR machines training. For the 6-SPS (Spherical–Prismatic–Spherical) model, the total training time was approximately 3 h. The off-line time for the 6-RRCRR model was about 9 h. Whereas the implementation of the method was mainly written in MATLAB, the random pose generator was coded in C++ and compiled as a MATLAB EXecutable (*MEX*) binary file to reduce the total computation time.

There are several libraries that implement SVMs, and one of the most widely used is LIBSVM (Chang and Lin, 2011). We have

chosen it for its ease of use, outstanding performance and because it can interface with MATLAB, which allows working with a rapid prototyping and testing framework. It is a library under active development, and provides a set of tools that can quickly yield acceptable results. It implements a set of SVM formulations, multi-class classification and probability estimates. Also, four basic kernel functions are available: linear, polynomial, sigmoid and Radial Basis Function (RBF) kernel, defined as

$$K(l_i, l_j) = e^{-\gamma \|l_i - l_j\|^2}, \quad \gamma > 0, \tag{9}$$

where the γ parameter behaves as a scaling factor. The LIBSVM library also provides a MATLAB interface with its functions, based on MATLAB *MEX*-files.

When exploring the best training settings for the SVR machines, we found that the Radial Basis Function kernel (9) yielded the best accuracy/training time ratio, especially as compared to the sigmoid kernel. Given the cost of adjusting each one individually, all the SVRs were trained with the same parameters. In the model training function, `svmtrain`, we have chosen the ε-SVM method, with the parameter $\gamma = 0.1$ (the width of the Gaussian basis function) and the cost equal to 1000 (C parameter in loss function). The given value C is a regularization parameter, which must be chosen to reflect the noise in the data (Gunn, 1998). It controls the trade-off between training error and model complexity. Also, we set the tolerance of the termination criterion equal to 5×10^{-3} and the epsilon parameter of the loss function equal to 10^{-3}. The forward kinematics is solved with the function `svmpredict`.

The resulting lookup table after the completion of the off-line phase occupied about 57 MB of memory for the 6-SPS robot, and about 35 MB for the 6-RRCRR model.

6.2. Generalized 6-SPS model

The method was first tested with the experimental model proposed by Liu et al. (1993), where the forward kinematics of a parallel manipulator is solved analytically. This allows us to verify the correctness and efficiency of the method against known solutions for a given platform. The top frame is an equilateral triangle, as can be seen in Fig. 6.

The model is described with respect to a pair of matrices, A and B, as shown in Fig. 7, where the coordinates of each vertex are described. Thus, both matrices are as follows:

$$A = \begin{bmatrix} \frac{\sqrt{3}}{6}a & \frac{1}{2}a & 0 \\ \frac{\sqrt{3}}{6}a & \frac{1}{2}a & 0 \\ -\frac{\sqrt{3}}{3}a & 0 & 0 \\ -\frac{\sqrt{3}}{3}a & 0 & 0 \\ \frac{\sqrt{3}}{6}a & -\frac{1}{2}a & 0 \\ \frac{\sqrt{3}}{6}a & -\frac{1}{2}a & 0 \end{bmatrix}^{\mathsf{T}}, \tag{10}$$

$$B = \begin{bmatrix} \frac{\sqrt{3}}{6}(2b+c) & \frac{1}{2}c & 0 \\ -\frac{\sqrt{3}}{6}(b-c) & \frac{1}{2}(b+c) & 0 \\ -\frac{\sqrt{3}}{3}(b+2c) & \frac{1}{2}b & 0 \\ -\frac{\sqrt{3}}{3}(b+2c) & -\frac{1}{2}b & 0 \\ -\frac{\sqrt{3}}{6}(b-c) & \frac{1}{2}(b+c) & 0 \\ \frac{\sqrt{3}}{6}(2b+c) & -\frac{1}{2}c & 0 \end{bmatrix}^{\mathsf{T}}. \tag{11}$$

Matrix A (10) indicates the positions of the links at the connection to the top platform relative to frame xyz with respect to the global frame XYZ. Similarly, matrix B (11) represents the links positions at the connection to the base relative to frame XYZ. The experimental model is described with the parameters

$$\begin{aligned} &a = 10.0\ \text{m} \quad b = 15.0\ \text{m} \quad c = 1.0\ \text{m} \\ &L_{min} = 8.0\ \text{m} \quad L_{max} = 15.0\ \text{m}, \end{aligned} \tag{12}$$

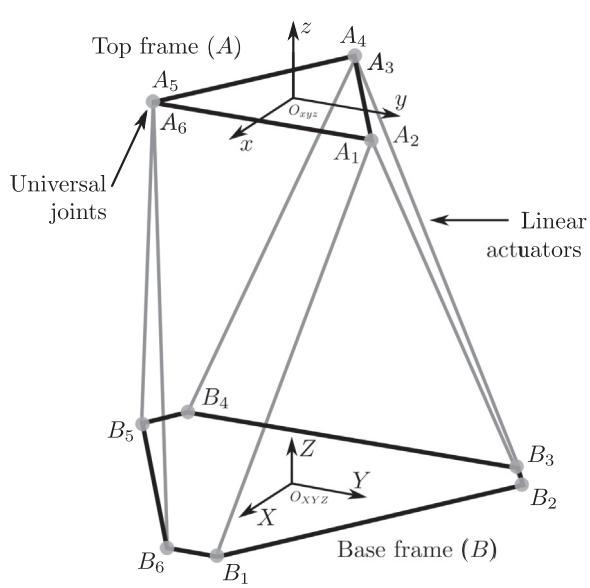

Fig. 6. 6-SPS model (Liu et al., 1993).

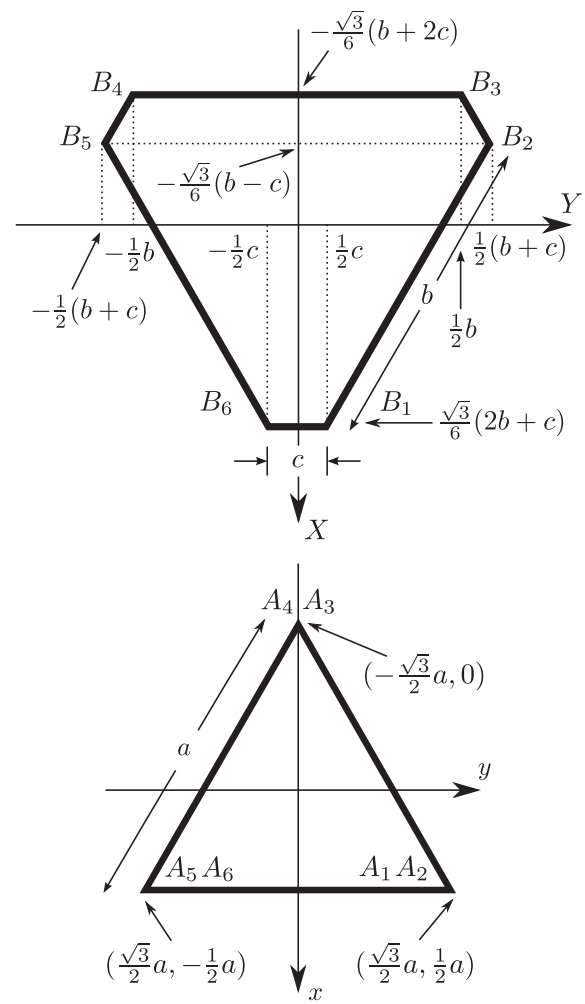

Fig. 7. 6-SPS model parameters: first—bottom frame; second—top frame.

where L_{min} and L_{max} are the minimum and maximum length of the linear actuators, respectively. It should be noted that Z coordinate on matrix A is zero because it describes the top frame w.r.t. the base coordinate system XYZ. For a given top platform posture, such plane is rotated and translated according to it.

The pose generation was carried out as described in Section 3, with random data generated within the ranges

$$\begin{aligned} x &= [-2\ \ 2] \quad (\mathrm{m}) \quad \alpha = [-60\ \ 60] \quad (^\circ) \\ y &= [-2.5\ \ 2] \quad (\mathrm{m}) \quad \beta = [-80\ \ 60] \quad (^\circ) \\ z &= [2\ \ 13] \quad (\mathrm{m}) \quad \gamma = [-70\ \ 70] \quad (^\circ), \end{aligned} \tag{13}$$

according to the extreme link configurations reported by Liu et al. (1993). The link space was decomposed into cells of length $lc = 3.5$ m in which applying (4) gives $N_i = 2$ divisions along each dimension. Therefore, the number of cells and regions is $2^6 = 64$. The number of poses generated was 320 000, about 4000 for each region.

There are six configurations of the linear actuators that correspond to representative positions of the top platform relevant when designing a parallel manipulator, since they provide an insight into its workspace. The *lowest and highest positions* of the end effector, with all the actuators at their minimum and maximum lengths, $l = [8, 8, 8, 8, 8, 8]^\top$ and $l = [15, 15, 15, 15, 15, 15]^\top$, respectively. The *most tilted position*, where one adjacent pair of linear actuators is at its maximum (or minimum) length, while the other two pairs are at their minimum (or maximum) lengths, with two asymmetric cases, $l = [15, 15, 8, 8, 8, 8]^\top$ and $l = [9.9015, 9.9015, 15, 15, 15, 15]^\top$. Finally, the *most twisted position*, where the three odd numbered actuators are at their minimum length with the even ones extended to their maximum, $l = [8, 15, 8, 15, 8, 15]^\top$, with a symmetric case $l = [15, 8,\ 15, 8, 15, 8]^\top$. In the experiments conducted, these extreme configurations are evaluated using the proposed approach and are compared with the other methods.

6.3. 6-RRCRR experimental model

This type of parallel robot is a Gough–Stewart platform with orthogonal non-intersecting RR pairs or offset U-joints configurations. Fig. 8 shows both frames and leg attachments. First joint on both frames is rotated by ϕ_A and ϕ_B with respect to X-axis, as shown in that figure. Legs are distributed radially every 120°. Such radius is P_A and P_B. The gap between two legs of the same pair is given by the angles θ_A and θ_B. They are also rotated by ρ_A and ρ_B to make them parallel. Legs are connected to the same attachment index, i.e., A_i is connected with B_i, with $i = 1, 2, 3, 4, 5, 6$. Finally, points B_i are within a plane located H_B millimeters with respect to Z-axis of the world frame. Similarly, end effector is H_A millimeters above the plane formed by points A_i.

The parameters of the experimental model used correspond to the M-850 hexapod by *Physik Instrumente*, which are summarized in

$$\begin{aligned} &H_B = 35.99275\ \mathrm{mm} \quad H_A = 25.0\ \mathrm{mm} \\ &P_B = 151.076\ \mathrm{mm} \quad P_A = 92.371\ \mathrm{mm} \\ &\theta_B = 0.2389\ \mathrm{rad} \quad \theta_A = 0.3922\ \mathrm{rad} \\ &\rho_B = 0.11945\ \mathrm{rad} \quad \rho_A = 0.1961\ \mathrm{rad} \\ &\phi_B = 0.32167\ \mathrm{rad} \quad \phi_A = 0\ \mathrm{rad} \\ &P_J = 13.0\ \mathrm{mm} \end{aligned} \tag{14}$$

For this model, random pose data was generated within the following ranges:

$$\begin{aligned} x &= [-50\ \ 50] \quad (\mathrm{mm}) \quad \alpha = [-15\ \ 15] \quad (^\circ) \\ y &= [-50\ \ 50] \quad (\mathrm{mm}) \quad \beta = [-15\ \ 15] \quad (^\circ), \\ z &= [303.4\ \ 353.4] \quad (\mathrm{mm}) \quad \gamma = [-30\ \ 30] \quad (^\circ) \end{aligned} \tag{15}$$

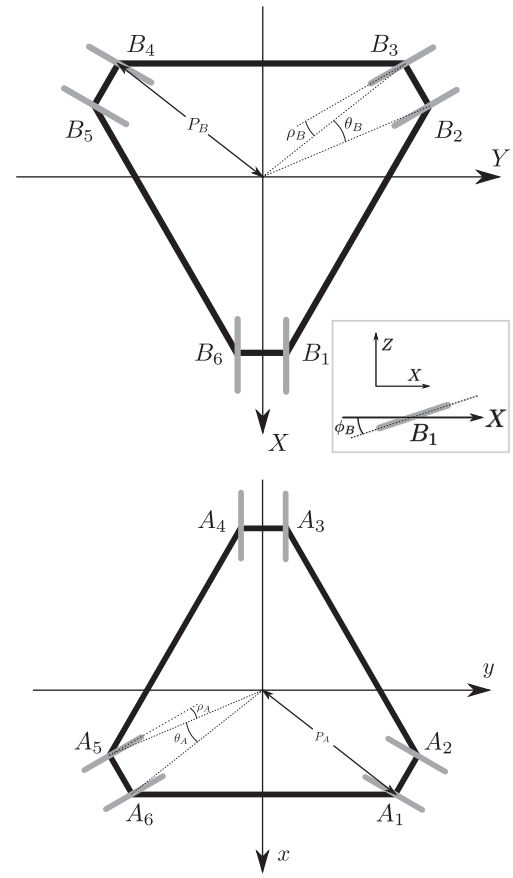

Fig. 8. 6-RRCRR model parameters: first—bottom frame; second—top frame.

as described by the datasheet and technical data, available at manufacturer's webpage (Physik Instrumente, PI). Similar to the 6-SPS model, the link space was decomposed into cells of size 120 mm, which gives $N_i = 2$ divisions along each dimension. The number of cells and regions was also $2^6 = 64$. Two accuracy tests were made, where each region were populated with about 2000 and 18 000 random poses. The total off-line time for the latter was almost 2 days, which had to be done only once.

6.4. Experimental results

Table 1 shows a comparison of the proposed approach and the polynomial method described by Tarokh (2007) applied to the 6-SPS parallel platform. In this experiment we solved the forward kinematics of extreme positions, and compared the yielded results against the exact analytical solution presented by Liu et al. (1993). This procedure allows us to check the correctness of the proposed method with the known forward kinematics solution given by the analytical method. The estimated link lengths obtained with the polynomial method for these configurations are not available, so we cannot compare the absolute full-range errors for both estimations. However, it can be seen that, while our method has in some cases a slight error, it actually yields a very good approximation of the exact solution. The best approximation seems to be the Most Tilted position #1, and the worst, the Highest position. The others yield more accurate estimations than those reported with the polynomial method. Results are presented with the correction step described in Section 5. The absolute full-range error for each extreme configuration is shown in Table 3, with and without correction. Each of them is lower than the average error reported by Tarokh (2007).

Table 1
Comparison of both methods and the analytical solution: 6-SPS model.

Method	x (m)	y (m)	z (m)	α (°)	β (°)	γ (°)
Lowest position						
A	0.0000	0.0000	2.6460	0.0000	0.0000	0.0000
P	0.0000	0.0000	2.6460	0.0000	0.0000	0.0000
S	0.0000	0.0000	2.6458	0.0006	0.0001	0.0000
Highest position						
A	0.0000	0.0000	12.9600	0.0000	0.0000	0.0000
P	0.0000	0.0000	12.9610	0.0000	0.0000	0.0000
S	0.0005	−0.0008	12.9610	0.0005	0.0009	−0.0016
Most Tilted position #1						
A	−1.2360	−2.1420	5.5030	58.9900	−73.8400	−46.0600
P	−1.2390	−2.1390	5.5060	58.9900	−74.0000	−46.0700
S	−1.2369	−2.1421	5.5036	58.9890	−73.8400	−46.0520
Most Tilted position #2						
A	−0.6930	−1.2000	10.4950	−47.7200	39.4200	−18.0100
S	−0.6939	−1.2017	10.4940	−47.7280	39.4280	−18.0160
Most Twisted position #1						
A	0.0000	0.0000	7.1920	0.0000	0.0000	68.3620
P	0.0000	0.0000	7.1940	0.0000	0.0000	68.3400
S	0.0001	0.0000	7.1924	0.0012	0.0000	68.3590
Most Twisted position #2						
A	0.0000	0.0000	7.1920	0.0000	0.0000	−68.3620
S	0.0000	0.0001	7.1923	−0.0001	−0.0010	−68.3600

A: Exact method (Liu et al., 1993).
P: Polynomial method (Tarokh, 2007).
S: SVR machine method proposed.

Table 2
Comparison of both methods: 6-RRCRR model.

Method	x (mm)	y (mm)	z (mm)	α (°)	β (°)	γ (°)
Test position #1						
T	55.0000	−32.0000	316.4000	−0.2094	0.4363	−0.3316
N	55.0001	−32.0000	343.4000	−0.2094	0.4363	−0.3316
S	54.9903	−31.9882	316.4018	−0.2095	0.4358	−0.3312
Test position #2						
T	−36.0000	44.0000	310.4000	0.0873	−0.2967	0.3665
N	−36.0000	44.0000	310.4000	0.0873	−0.2967	0.3665
S	−36.0020	43.9994	310.3986	0.0872	−0.2966	0.3666

T: Test position.
N: Numerical method (Dalvand and Shirinzadeh, 2012).
S: SVR machine method proposed.

Table 3
Full-range errors for extreme configurations: 6-SPS model.

Correction	**Lowest**	**Highest**
w/o	0.0142%	0.0137%
w/	0.0007%	0.0048%
Correction	**Most Tilted #1**	**Most Tilted #2**
w/o	0.0105%	0.0195%
w/	0.0098%	0.0067%
Correction	**Most Twisted #1**	**Most Twisted #2**
w/o	0.0211%	0.0202%
w/	0.0007%	0.0009%

The next experiment is an accuracy test, where 1000 randomly generated link lengths are solved with the proposed method. Then, the inverse kinematics of those estimated poses is solved applying (1). The average absolute error of the estimated link lengths is obtained with $|l-\hat{l}|$. Also, this test is useful to measure the on-line evaluation time. Since it is very difficult to take a measure of the total evaluation time for a single configuration of the linear actuators, a more appropriate approach is to obtain the average from a sequence of tests. For the 6-SPS parallel platform, the average link length error and its standard deviation σ are lower after the correction step, as seen in Table 4. The total time taken to complete the accuracy test was 4.87 s, which means an average evaluation time of 4.87 ms for each test.

Table 4
Accuracy test with 1000 link configurations: 6-SPS model.

Method	**Error**	σ	
P	0.00478 m 0.6829%	0.0089 m 0.1271%	**w/o correction**
S	0.1929 m 2.7556%	0.5152 m 7.3597%	
P	0.00124 m 0.0177%	0.0038 m 0.0543%	**w/correction**
S	0.0008 m 0.0117%	0.0022 m 0.0319%	

P: Polynomial method (Tarokh, 2007).
S: SVR machine method proposed.

Table 5
Accuracy test with 1000 link configurations: 6-RRCRR model.

Experiment	**Error**	σ	
Test #1	1.7115 mm 0.7131%	1.8619 mm 0.7758%	**w/o correction**
Test #2	0.0841 mm 0.0350%	0.0357 mm 0.0149%	
Test #1	0.0083 mm 0.0035%	0.0125 mm 0.0052%	**w/correction**
Test #2	0.0011 mm 0.0005%	0.0010 mm 0.0004%	

SVR machine method proposed.
Test #1: About 2000 poses per region.
Test #2: About 18 000 poses per region.

Similarly, we solved the forward kinematics for the poses shown in Table 2, two end effector positions proposed by Dalvand and Shirinzadeh (2012). It can be seen that the proposed method found the solution with a slight error. However, it should be noted that our method is able to find more than one solution when it exists and if the SVR machines are trained with enough data points. Furthermore, the pose prediction using SVR machines is a fast and reliable process, i.e., it does not need an initial guess, and does not suffer from convergence problems. However, this method is able to yield very accurate solutions, which can be further improved with more data points and fine tuning the training parameters, which imply more off-line training time, as shown in Table 5. Unfortunately, there is no similar experiment for the accuracy test to compare with the method proposed by Dalvand and Shirinzadeh (2012) for the 6-RRCRR model. However, results showed a very low error, further improved by the correction step. It can be seen that highly populated regions, i.e., more training data for each SVR machine, yield more accurate results after this test, with a tradeoff of more training time.

Finally, a performance comparison has been conducted on the 6-SPS model. Table 6 shows that accuracy can be improved by increasing the parameter C, although the training time increases as well. However, there is a risk of overfitting when its value is too high. Furthermore, γ affects the accuracy more clearly, since it leads to a change in the model.

Table 6
Performance comparison with different parameters.

C	γ	Time (s)	F-R Error	σ
100	10^{-1}	3092.2	0.0191% 0.0013 m	0.0467% 0.0033 m
500	10^{-1}	7066.63	0.0126% 0.0009 m	0.0347% 0.0024 m
1000	10^{-1}	11255.1	0.0117% 0.0008 m	0.0319% 0.0022 m
2000	10^{-1}	18075.5	0.0117% 0.0008m	0.0320% 0.0022 m
500	10^{0}	2534.92	0.7864% 0.0550 m	1.8259% 0.1278 m
500	10^{-2}	2819.31	0.0324% 0.0023 m	0.0634% 0.0044 m
500	10^{-3}	2137.57	0.1565% 0.0110 m	0.1562% 0.0109 m
1000	10^{-3}	2196.1	0.1358% 0.0095 m	0.1402% 0.0098 m

7. Conclusion

In this paper we have proposed an alternative approach for solving the forward kinematics problem of parallel robots, a very popular class of manipulators that has gained interest in a wide variety of engineering applications. Unlike other methods, the proposed algorithm is not dependent on the geometry of the platform. It consists of a preprocessing phase and a fast evaluation step: random generated poses are validated and classified, in order to be used as training samples of a popular machine learning method, the Support Vector Machines. It can also be applied to problems that require high accuracy, without high memory requirements.

The main contributions of the proposed method are as follows. Firstly, since divisions of the link space (number of cells) are relatively few, the random pose generation is fast. Secondly, the classification takes advantage of fitted Gaussian Mixture Models, which allows simultaneous estimation of the number of clusters and classification. Finally, using Support Vector Regression machines yields a very good approximation of the analytical solution, which is further improved by a correction step. This also leads to an evaluation phase reasonably fast enough to be applied to real-time applications.

Some other considerations are also worthy of note. For example, a smaller cell size would provide higher accuracy, but the time needed to populate the regions with valid poses would increase as well. Finally, the accuracy of the solutions can also be easily improved by adjusting the parameters of the SVRs, at the cost of more off-line training time.

Acknowledgments

The authors gratefully acknowledge the contribution of the Spanish Ministry of Economy and Competitiveness under Project SAGENIA DPI2010-18349.

References

Akaike, H., 1974. A new look at the statistical model identification. IEEE Trans. Autom. Control 19, 716–723.
Boser, B.E., Guyon, I., Vapnik, V., 1992. A training algorithm for optimal margin classifiers. In: Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory, ACM Press, pp. 144–152.
Chang, C.C., Lin, C.J., 2011. LIBSVM: a library for support vector machines. ACM Trans. Intell. Syst. Technol. 2, 27:1–27:27. Software available at ⟨http://www.csie.ntu.edu.tw/~cjlin/libsvm⟩.
Chen, S.H., Fu, L.C., 2006. The forward kinematics of the 6–6 Stewart platform using extra sensors. In: IEEE International Conference on Systems, Man and Cybernetics, 2006 (SMC '06), pp. 4671–4676.
Chiddarwar, S.S., Babu, N.R., 2010. Comparison of RBF and MLP neural networks to solve inverse kinematic problem for 6R serial robot by a fusion approach. Eng. Appl. Artif. Intell. 23, 1083–1092.
Clavel, R., 1988. DELTA, a fast robot with parallel geometry. In: Burckhardt, C.W. (Ed.), Proceedings of the 18th International Symposium on Industrial Robots. Springer-Verlag, pp. 91–100.
Collins, C.L., Long, G.L., 1995. On the duality of twist/wrench distributions in serial and parallel chain robot manipulators. In: IEEE International Conference on Robotics and Automation, 1995, pp. 526–531.
Dalvand, M.M., Shirinzadeh, B., 2011. Forward kinematics analysis of offset 6-RRCRR parallel manipulators. Proc. Inst. Mech. Eng. C: J. Mech. Eng. Sci. 255, 3011–3018.
Dalvand, M.M., Shirinzadeh, B., 2012. Kinematics analysis of 6-DOF parallel micromanipulators with offset U-joints: a case study. Int. J. Intell. Mechatronics Robot. 2, 28–40.
Dasgupta, B., Mruthyunjaya, T.S., 2000. The Stewart platform manipulator: a review. Mech. Mach. Theory 35, 15–40.
Dempster, A.P., Laird, N.M., Rubin, D.B., 1977. Maximum likelihood estimation from incomplete data via the EM algorithm. J. R. Stat. Soc. Ser. B 39, 1–38.
Dieudonne, J.E., Parrish, R.V., Bardusch, R.E., 1972. An Actuator Extension Transformation for a Motion simulator and an inverse transformation applying Newton–Raphson's method. Technical Report NASA-TN-D-7067. NASA Langley Research Center, Hampton, VA.
Gao, Z., Zhang, D., Hu, X., Ge, Y., 2010. Design, analysis, and stiffness optimization of a three degree of freedom parallel manipulator. Robotica 28, 349–357.
Gough, V.E., 1956–1957. Contribution to discussion of papers on research in automobile stability, control and tyre performance. In: Proceedings of the Auto Divisional Institute of Mechanical Engineers, pp. 392–394.
Gunn, S.R., 1998. Support Vector Machines for Classification and Regression. Technical Report. School of Electronics and Computer Science, University of Southampton, Southampton, UK.
Karlik, B., Aydin, S., 2000. An improved approach to the solution of inverse kinematics problems for robot manipulators. Eng. Appl. Artif. Intell. 13, 159–164.
Köker, R., 2005. Reliability-based approach to the inverse kinematics solution of robots using Elman's networks. Eng. Appl. Artif. Intell. 18, 685–693.
Liu, K., Fitzgerald, J.M., Lewis, F.L., 1993. Kinematic analysis of a Stewart platform manipulator. IEEE Trans. Ind. Electron. 40, 282–293.
McCallion, H., Pham, D., 1979. The analysis of a six degrees of freedom work station for mechanized assembly. In: 5th World Congress on Theory of Machines and Mechanisms, Montréal, pp. 611–616.
Merlet, J.P., 1993. Closed-form resolution of the direct kinematics of parallel manipulators using extra sensors data. In: Proceedings of IEEE International Conference on Robotics and Automation, 1993, pp. 200–204.
Merlet, J.P., 2004. Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis. Int. J. Robot. Res. 23 (3), 221–235.
Nanua, P., Waldron, K., Murthy, V., 1990. Direct kinematic solution of a Stewart platform. IEEE Trans. Robot. Autom. 6, 438–444.
Panneton, F., L'Ecuyer, P., Matsumoto, M., 2006. Improved long-period generators based on linear recurrences modulo 2. ACM Trans. Math. Softw. 32, 1–16.
Parikh, P.J., Lam, S.S., 2009. Solving the forward kinematics problem in parallel manipulators using an iterative artificial neural network strategy. Int. J.Adv. Manuf. Technol. 40, 595–606.
Peña, D., Prieto, F.J., 2001. Multivariate outlier detection and robust covariance matrix estimation. Technometrics 43, 286–310.
Physik Instrumente, PI. M-850 Hexapod, Technical data available at ⟨http://www.physikinstrumente.com/micropositioningsystems/8_4t.html⟩.
Pollard, W.L.G., 1940. Spray painting machine.
Siciliano, B., 1999. The Tricept robot: inverse kinematics, manipulability analysis and closed-loop direct kinematics algorithm. Robotica 17, 437–445.
Stewart, D., 1965–1966. A platform with six degrees of freedom. Proc. Inst. Mech. Eng. 180, 171–186.
Tarokh, M., 2007. Real time forward kinematics solutions for general Stewart platforms. In: IEEE International Conference on Robotics and Automation, 2007, pp. 901–906.
Vapnik, V., 1995. The Nature of Statistical Learning Theory. Springer-Verlag New York, Inc., New York, NY, USA.
Vapnik, V., Golowich, S.E., Smola, A., 1996. Support vector method for function approximation, regression estimation, and signal processing. In: Advances in Neural Information Processing Systems 9. MIT Press, pp. 281–287.